

✓ Trabajo Práctico Final: Proyecto Integrador - Grupo

Integrantes:

- Gabriel Dannunzio
- Juan Manuel Etchehun
- Pablo Tapia
- Pablo Donato Nuñez

Descripción:

El dataset a trabajar pertenece a un equipo de analistas de datos de una cadena de supermercado en USA durante el período de 3 años. El objetivo es realizar un Análisis Exploratorio de Datos (EDA) para identificar patrones de compra, analizar las ventas y extraer insights valiosos para la empresa.

Dataset: Supermarket_Sales

Objetivos:

- Limpiar y preprocesar el dataset
- Obtener estadísticas descriptivas y analizarlas.
- Visualizar patrones
- Extraer conclusiones
- Presentar conclusiones con objetivo de potenciar el negocio
- La Visualización deberá presentar con PowerBI para explicitar la propuesta al negocio

✓ Preparación del entorno de trabajo → Herramientas que usaremos

```
1 # Librerías principales para análisis y manipulación de datos
2 import pandas as pd
3 import numpy as np
```

```
1 # Librerías para visualización de datos
2 import matplotlib.pyplot as plt
3 import seaborn as sns
```

```
1 # Herramientas auxiliares para limpieza de texto y manejo de rutas
2 import re
3 from pathlib import Path
4 from difflib import get_close_matches
```

```
1 # Configuración para mostrar números con dos decimales
2 pd.options.display.float_format = '{:.2f}'.format
```

```
1 # Estilo visual para los gráficos
2 sns.set_theme(style='whitegrid')
```

✓ 1. Carga del dataset

Primero cargamos el archivo crudo desde Google Drive.

Como ya tenemos identificada la carpeta del proyecto, usamos la ruta directa al archivo `df_sales_crudo.csv`.

La base se carga en una variable llamada `df_raw`, porque representa el dataset original, antes de aplicar limpieza o transformaciones.

```
1 # link del CSV de Drive
2 url = "https://drive.google.com/file/d/1iBI1Q42QQjsi17PYuhLx04ZEXUSoAFay/view?usp=sharing"
3
4 # Convertir link de Drive a descarga directa
5 file_id = url.split("/d/")[1].split("/")[0]
6 url_descarga = f"https://drive.google.com/uc?id={file_id}"
7
8 # Leer CSV
9 df_raw = pd.read_csv(url_descarga)
```

```
10
11 print("Dataset cargado correctamente")
12 print("Dimensión:", df_raw.shape)
13
14 df_raw.head()
```

Dataset cargado correctamente
Dimensión: (9994, 21)

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Cat
0	1	CA-2016-152156	2016-11-08 00:00:00	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-BO-10001798	Fu
1	2	CA-2016-152156	2016-11-08 00:00:00	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Fu

2. Primera mirada general

Antes de tocar el dataset, revisamos tamaño, columnas y tipos de datos. Esto ayuda a detectar problemas de entrada.

```
1 print('Filas y columnas', df_raw.shape)
2 df_raw.info()
```

Filas y columnas (9994, 21)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
Column Non-Null Count Dtype

0 Row ID 9994 non-null int64
1 Order ID 9994 non-null object
2 Order Date 9994 non-null object
3 Ship Date 9994 non-null object
4 Ship Mode 9767 non-null object
5 Customer ID 9994 non-null object
6 Customer Name 9773 non-null object
7 Segment 9771 non-null object
8 Country 9994 non-null object
9 City 9761 non-null object
10 State 9775 non-null object
11 Postal Code 9994 non-null int64
12 Region 9790 non-null object
13 Product ID 9994 non-null object
14 Category 9798 non-null object
15 Sub-Category 9763 non-null object
16 Product Name 9776 non-null object
17 Sales 9994 non-null object
18 Quantity 9994 non-null object
19 Discount 9994 non-null float64
20 Profit 9994 non-null float64
dtypes: float64(2), int64(2), object(17)
memory usage: 1.6+ MB

```
1 df_raw.head(10)
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Cat
0	1	CA-2016-152156	2016-11-08 00:00:00	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-BO-10001798	Fu
1	2	CA-2016-152156	2016-11-08 00:00:00	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Fu
2	3	CA-2016-138688	2016-06-12 00:00:00	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	90036	West	OFF-LA-10000240	Of
3	4	US-2016-181223	2016-01-09 00:00:00	2016-01-13	Standard Class	CG-13522	Shane	Consumer	United States	San Jose	...	95128	North	OFF-CA-10000240	Of
4	5	US-2016-181223	2016-01-09 00:00:00	2016-01-13	Standard Class	CG-13522	Shane	Consumer	United States	San Jose	...	95128	North	OFF-CA-10000240	Of
5	6	US-2016-181223	2016-01-09 00:00:00	2016-01-13	Standard Class	CG-13522	Shane	Consumer	United States	San Jose	...	95128	North	OFF-CA-10000240	Of
6	7	US-2016-181223	2016-01-09 00:00:00	2016-01-13	Standard Class	CG-13522	Shane	Consumer	United States	San Jose	...	95128	North	OFF-CA-10000240	Of
7	8	US-2016-181223	2016-01-09 00:00:00	2016-01-13	Standard Class	CG-13522	Shane	Consumer	United States	San Jose	...	95128	North	OFF-CA-10000240	Of
8	9	US-2016-181223	2016-01-09 00:00:00	2016-01-13	Standard Class	CG-13522	Shane	Consumer	United States	San Jose	...	95128	North	OFF-CA-10000240	Of
9	10	US-2016-181223	2016-01-09 00:00:00	2016-01-13	Standard Class	CG-13522	Shane	Consumer	United States	San Jose	...	95128	North	OFF-CA-10000240	Of

Diagnóstico de valores nulos y duplicados

```
1 resumen_nulos = (  
2     df_raw.isna().sum()  
3     .to_frame('nulos')  
4     .assign(porcentaje=lambda x: (x['nulos'] / len(df_raw) * 100).round(2))  
5     .sort_values('nulos', ascending=False)  
6 )  
7  
8 resumen_nulos
```

	nulos	porcentaje
City	233	2.33
Sub-Category	231	2.31
Ship Mode	227	2.27
Segment	223	2.23
Customer Name	221	2.21
State	219	2.19
Product Name	218	2.18
Region	204	2.04
Category	196	1.96
Ship Date	0	0.00
Order Date	0	0.00
Order ID	0	0.00
Row ID	0	0.00
Postal Code	0	0.00
Country	0	0.00
Customer ID	0	0.00
Product ID	0	0.00
Sales	0	0.00
Quantity	0	0.00
Discount	0	0.00
Profit	0	0.00

Pasos siguientes:

[Generar código con resumen_nulos](#)[New interactive sheet](#)

```
1 print('Filas duplicadas exactas:', df_raw.duplicated().sum())
```

Filas duplicadas exactas: 0

Observación inicial

En esta primera revisión ya aparecen algunas cosas para trabajar:

- `Sales` y `Quantity` figuran como texto, aunque deberían ser numéricas.
- `Order Date` y `Ship Date` todavía no están como fechas.
- Hay nulos en columnas como `City`, `State`, `Region`, `Category`, `Segment`, `Ship Mode`, etc.
- `Postal Code` aparece como número, pero en realidad es un código/identificador geográfico.
- Hay errores de tipeo en categorías de texto, por ejemplo en `Ship Mode`, `Segment` y `Category`.

3. Revisión de problemas puntuales

Ahora miramos algunos problemas concretos antes de limpiarlos. Esto sirve para justificar el preprocesamiento.

```
1 # Revisamos si algunas columnas importantes pueden convertirse al tipo de dato esperado.  
2 # Sales y Quantity deberían ser numéricas.  
3 # Order Date y Ship Date deberían ser fechas.  
4  
5 problemas_conversion = pd.DataFrame({  
6     'columna': ['Sales', 'Quantity', 'Order Date', 'Ship Date'],
```

```

7     'valores_problematicos': [
8         pd.to_numeric(df_raw['Sales'], errors='coerce').isna().sum(),
9         pd.to_numeric(df_raw['Quantity'], errors='coerce').isna().sum(),
10        pd.to_datetime(df_raw['Order Date'], errors='coerce').isna().sum(),
11        pd.to_datetime(df_raw['Ship Date'], errors='coerce').isna().sum()
12    ]
13 })
14
15 problemas_conversion

```

	columna	valores_problematicos	
0	Sales	50	
1	Quantity	50	
2	Order Date	30	
3	Ship Date	0	

Pasos siguientes:

[Generar código con problemas_conversion](#)

[New interactive sheet](#)

```

1 # Mostramos ejemplos concretos de valores que no pueden convertirse al tipo esperado.
2 # Esto ayuda a entender qué errores existen antes de realizar la limpieza.
3
4 print('Ejemplos problemáticos en Sales:')
5 display(df_raw.loc[pd.to_numeric(df_raw['Sales'], errors='coerce').isna(), ['Sales']].drop_duplicates().head())
6
7 print('Ejemplos problemáticos en Quantity:')
8 display(df_raw.loc[pd.to_numeric(df_raw['Quantity'], errors='coerce').isna(), ['Quantity']].drop_duplicates().head())
9
10 print('Ejemplos problemáticos en Order Date:')
11 display(df_raw.loc[pd.to_datetime(df_raw['Order Date'], errors='coerce').isna(), ['Order Date']].drop_duplicates().head())
12

```

Ejemplos problemáticos en Sales:

Sales

43 Corrupted_Sale

Ejemplos problemáticos en Quantity:

Quantity

15 Invalid_Qty

Ejemplos problemáticos en Order Date:

Order Date

629 DATE_ERROR_FORMAT

✓ Caso especial: Postal Code

Postal Code parece numérico, pero no lo es en sentido analítico. No tiene sentido calcular un promedio de códigos postales. Además, en Estados Unidos algunos códigos pueden empezar con cero, y si se guardan como número se pierde ese cero inicial.

```

1 # Postal Code se analiza como identificador geográfico, no como número.
2 # Revisamos la longitud de los códigos para detectar casos que pudieron perder un cero inicial.
3
4
5 longitud_postal_antes = df_raw['Postal Code'].astype(str).str.len().value_counts().sort_index()
6 longitud_postal_antes

```

	count
Postal Code	
4	449
5	9545

dtype: int64

```

1 # Mostramos algunos ejemplos de códigos de 4 dígitos antes de corregirlos.
2 postal_4_digitos = df_raw.loc[df_raw['Postal Code'].astype(str).str.len() == 4, 'Postal Code'].head(10)
3 postal_4_digitos

```

Postal Code	
185	6824
197	7090
267	7960
298	7109
299	7109
300	7109
301	7109
302	7109
306	8701
307	8701

dtype: int64

4. Limpieza y transformación

Creemos una copia para no modificar el dataset original. El criterio será:

1. convertir ventas, cantidades y fechas;
2. tratar `Postal Code` como texto de 5 caracteres;
3. normalizar algunas columnas categóricas;
4. eliminar solo filas con problemas en campos críticos;
5. completar nulos categóricos como `Sin dato`, sin inventar información.

1 Empieza a programar o a crear código con IA.

```
1 df = df_raw.copy()
2
3 # Conversiones numéricas
4 # Si aparece algo como Corrupted_Sale o Invalid_Qty, lo pasamos a NaN para tratarlo después.
5 df['Sales'] = pd.to_numeric(df['Sales'], errors='coerce')
6 df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')
7
8 # Conversiones de fecha
9 df['Order Date'] = pd.to_datetime(df['Order Date'], errors='coerce')
10 df['Ship Date'] = pd.to_datetime(df['Ship Date'], errors='coerce')
11
12 # Postal Code como texto. zfill(5) agrega ceros a la izquierda cuando corresponde.
13 df['Postal Code'] = df['Postal Code'].astype(str).str.zfill(5)
14
15 # Muestro los tipos de datos después de la conversión
16 print(df[['Sales', 'Quantity', 'Order Date', 'Ship Date', 'Postal Code']].dtypes)
```

```
Sales          float64
Quantity       float64
Order Date     datetime64[ns]
Ship Date      datetime64[ns]
Postal Code     object
dtype: object
```

```
1 # Verificación de Postal Code después del ajuste
2 print(df['Postal Code'].astype(str).str.len().value_counts().sort_index())
3 df[['Postal Code']].head()
```

```
Postal Code
5      9994
Name: count, dtype: int64
```

Postal Code	
0	42420
1	42420
2	90036
3	33311
4	33311

```
1 # Convertimos las columnas numéricas.
2 # Los valores inválidos se transforman en NaN para poder tratarlos después.
```

```

3 # errors='coerce' transforma los valores corruptos en NaN así no rompe código
4
5 df['Sales'] = pd.to_numeric(df['Sales'], errors='coerce')
6 df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')

```

Normalización de variables categóricas

En `Ship Mode`, `Segment` y `Category` hay errores de tipeo. Para no hacer un diccionario enorme a mano, uso una función simple que compara contra valores válidos esperados.

No es una solución perfecta para producción, pero para este análisis sirve porque conocemos las categorías correctas

```

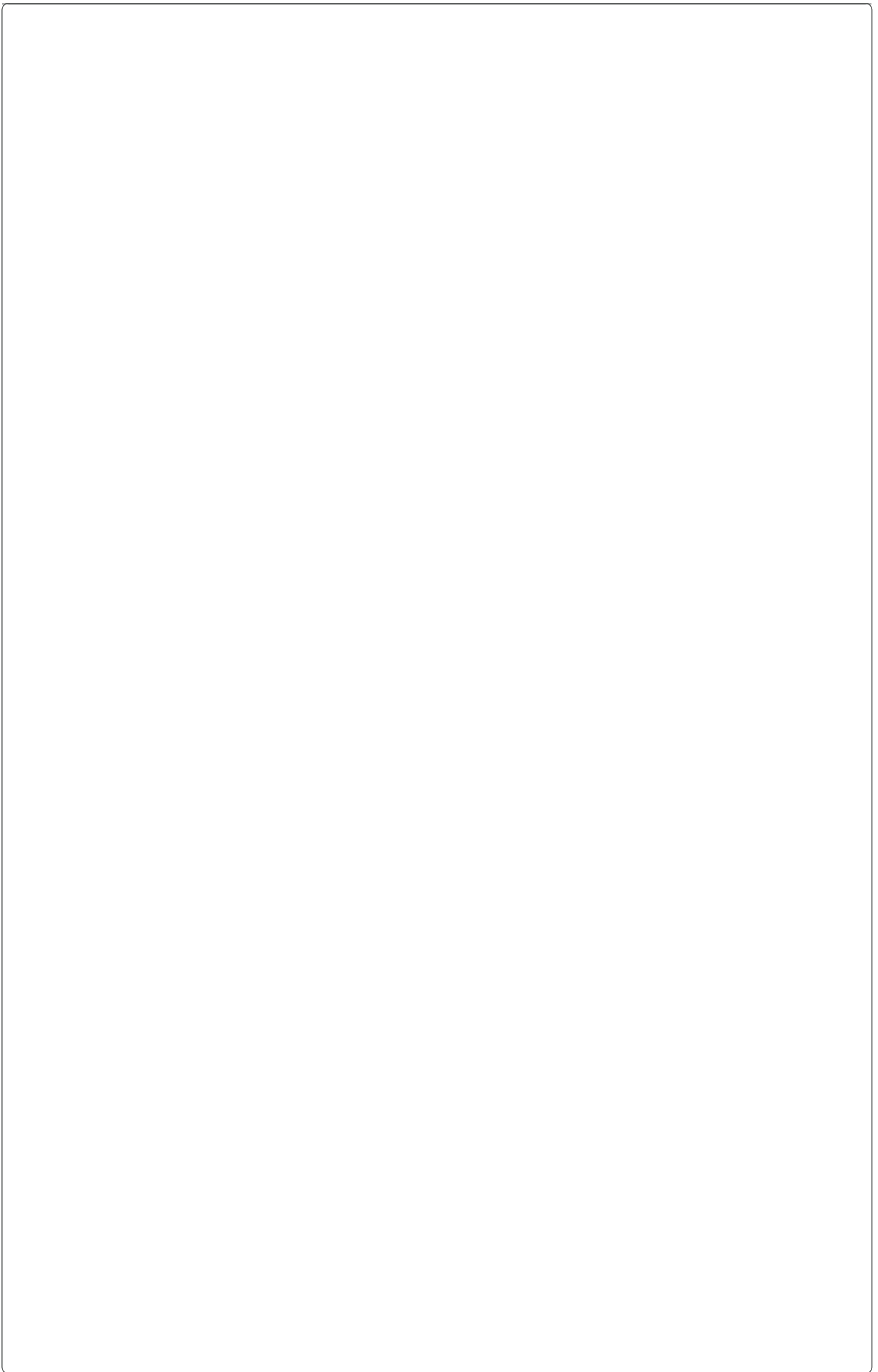
1 # Función auxiliar para normalizar textos categóricos.
2 # Corrige espacios, mayúsculas/minúsculas y algunos errores de tipeo simples
3 # comparando contra una lista de valores válidos.
4
5 def normalizar_texto(valor, valores_validos, cutoff=0.70):
6     if pd.isna(valor):
7         return np.nan
8
9     texto = str(valor).lower().strip()
10    texto = re.sub(r'\s+', ' ', texto)
11
12    valores_norm = [v.lower() for v in valores_validos]
13
14    if texto in valores_norm:
15        return valores_validos[valores_norm.index(texto)]
16
17    parecido = get_close_matches(texto, valores_norm, n=1, cutoff=cutoff)
18    if parecido:
19        return valores_validos[valores_norm.index(parecido[0])]
20
21    return texto.title()

```

```

1 # Definimos los valores correctos esperados para algunas columnas categóricas.
2 # Luego revisamos cómo aparecen esos valores en la base cruda antes de limpiarlos.
3
4
5 # Valores válidos esperados según el dataset original de ventas
6 ship_validos = ['Standard Class', 'Second Class', 'First Class', 'Same Day']
7 segment_validos = ['Consumer', 'Corporate', 'Home Office']
8 category_validos = ['Furniture', 'Office Supplies', 'Technology']
9
10 # Antes
11 display(df_raw['Ship Mode'].value_counts(dropna=False).head(15).to_frame('antes_ship_mode'))
12 display(df_raw['Segment'].value_counts(dropna=False).head(15).to_frame('antes_segment'))
13 display(df_raw['Category'].value_counts(dropna=False).head(15).to_frame('antes_category'))

```



antes_ship_mode 

```
1 # Aplicamos la función de normalización a columnas categóricas.
2 # Luego revisamos nuevamente los valores para comparar el antes y el después.
3
4
5 df['Ship Mode'] = df['Ship Mode'].apply(lambda x: normalizar_texto(x,
    ship_validos, cutoff=0.70))
6 df['Segment'] = df['Segment'].apply(lambda x: normalizar_texto(x,
    segment_validos, cutoff=0.70))
7 df['Category'] = df['Category'].apply(lambda x: normalizar_texto(x,
    category_validos, cutoff=0.68))
8
9 # Después
10 display(df['Ship Mode'].value_counts(dropna=False).to_frame(
    ('despues_ship_mode'))
11 display(df['Segment'].value_counts(dropna=False).to_frame('despues_segment'))
12 display(df['Category'].value_counts(dropna=False).to_frame('despues_category'))
```

Standard Class despues_ship_mode 

Standard Class	2
Standard Class	5844
Second Class	1907
First Class	1491
Same Day	antes_segment 525
NaN	227

Consumer despues_segment

Consumer	2932
----------	------

Consumer 5077

Consumer 2950

Home Office 1744

Consumer 243

Consumer despues_category

Consumer	3
----------	---

Office Supplies 5906

Consumer 3 2081

Technology 1811

Home Office 3 196

Consumer 2

Tratamiento de nulos

Consumer 2

Para este primer avance vamos a usar un criterio simple:

- si faltan (Sales, Quantity, Order Date o Ship Date), la fila no sirve para análisis principal de ventas y se elimina;
- si falta un dato categórico, no lo inventamos: lo marcamos como (Sin dato).

Office Supplies 5868

Esto permite conservar la mayoría de los registros sin generar datos falsos.

Furniture 2061

```
1 # Eliminamos filas con valores faltantes en campos críticos para el análisis.
2 # No eliminamos cualquier nulo del dataset, solo aquellos que afectan directamente
3 # las ventas, cantidades, fechas o identificadores principales.
4 # Además, medimos cuántas filas se pierden para evaluar el impacto de esta decisión.
5
6 filas_antes = len(df)
7
8 campos_criticos = ['Sales', 'Quantity', 'Order Date', 'Ship Date', 'Order ID', 'Product ID']
9 df_clean = df.dropna(subset=campos_criticos).copy()
10
11 filas_despues = len(df_clean)
12 print('Filas antes:', filas_antes)
13 print('Filas después de quitar críticos nulos:', filas_despues)
14 print('Filas eliminadas:', filas_antes - filas_despues)
15 print('Porcentaje eliminado:', round((filas_antes - filas_despues) / filas_antes * 100, 2), '%')
```

Filas antes: 9994
 Furniture 2
 Office Supplies 2
 Filas después de quitar críticos nulos: 9864

Filas eliminadas: 130
Office Supplies
Porcentaje eliminado: 1.3 % 2

```
1
2 # En esta etapa limpiamos las columnas de texto:
3 # - eliminamos espacios al inicio y al final;
4 # - reemplazamos valores faltantes por "Sin dato";
5 # - convertimos Quantity a entero porque representa unidades vendidas.
6 # Finalmente revisamos si todavía quedan valores nulos en la base.
7
8 # Limpiamos espacios en columnas de texto y completamos nulos como Sin dato
9 columnas_texto = df_clean.select_dtypes(include=['object', 'string']).columns
10
11 for col in columnas_texto:
12     df_clean[col] = df_clean[col].astype('string').str.strip()
13     df_clean[col] = df_clean[col].fillna('Sin dato')
14
15 # Quantity debería ser entera porque representa unidades
16 # Usamos Int64 por seguridad, aunque luego no deberían quedar nulos.
17 df_clean['Quantity'] = df_clean['Quantity'].astype('Int64')
18
19 print('Nulos restantes:', df_clean.isna().sum().sum())
20 df_clean.head()
```

Nulos restantes: 0

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region	Product ID	Category
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-BO-10001798	Furniture
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	South	FUR-CH-10000454	Furniture

5. Variables derivadas

Agregamos algunas columnas útiles para el análisis:

- días de envío;
- año;
- mes;
- año-mes;
- margen aproximado de ganancia sobre ventas.

```
1 # Síntesis
2
3 # Creamos variables derivadas para enriquecer el análisis:
4 # - Shipping Days mide la diferencia entre fecha de envío y fecha de pedido.
5 # - Year, Month y Order Year-Month permiten analizar la evolución temporal.
6 # - Profit Margin calcula la rentabilidad relativa de cada venta.
7 # También revisamos los valores mínimos y máximos de Shipping Days para detectar fechas inconsistentes.
8
9 df_clean['Shipping Days'] = (df_clean['Ship Date'] - df_clean['Order Date']).dt.days
10
11 df_clean['Year'] = df_clean['Order Date'].dt.year
12 df_clean['Month'] = df_clean['Order Date'].dt.month
13 df_clean['Month Name'] = df_clean['Order Date'].dt.month_name()
14 df_clean['Order Year-Month'] = df_clean['Order Date'].dt.to_period('M').astype(str)
15
16 # Profit Margin: ganancia / ventas. Si Sales fuera 0, evitamos división por cero.
17 df_clean['Profit Margin'] = np.where(
18     df_clean['Sales'] != 0,
19     df_clean['Profit'] / df_clean['Sales'],
20     np.nan
21 )
22
23 # Control de posibles fechas raras
24 print('Shipping Days mínimos y máximos:')
25 print(df_clean['Shipping Days'].agg(['min', 'max']))
26
27 df_clean[['Order Date', 'Ship Date', 'Shipping Days', 'Year', 'Month', 'Order Year-Month', 'Profit Margin']].head()
```

```
Shipping Days mínimos y máximos:
min    0
max    7
Name: Shipping Days, dtype: int64
```

	Order Date	Ship Date	Shipping Days	Year	Month	Order Year-Month	Profit Margin	
0	2016-11-08	2016-11-11	3	2016	11	2016-11	0.16	
1	2016-11-08	2016-11-11	3	2016	11	2016-11	0.30	
2	2016-06-12	2016-06-16	4	2016	6	2016-06	0.47	
3	2015-10-11	2015-10-18	7	2015	10	2015-10	-0.40	
4	2015-10-11	2015-10-18	7	2015	10	2015-10	0.11	

```
1 # Si apareciera algún envío con días negativos, se puede revisar aparte.
2 # En este caso lo dejamos chequeado. Por el caso de que aparezcan casos raros
3 # Inconsistencia→ porque la fecha de envío no debería ser anterior
4 # a la fecha del pedido.
5
6 df_clean[df_clean['Shipping Days'] < 0].head()
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Sales	Quantity	Discount	Profit	Shipping Days
--------	----------	------------	-----------	-----------	-------------	---------------	---------	---------	------	-----	-------	----------	----------	--------	---------------

6. Dataset limpio para análisis y visualización

En esta etapa generamos una versión final del dataset limpio, preparada para ser utilizada posteriormente en herramientas de visualización como **Power BI** o **Looker Studio**.

Antes de exportar el archivo, estandarizamos la cantidad de decimales en las columnas numéricas principales. Esto permite trabajar con valores más consistentes, facilita la lectura de los datos y evita diferencias innecesarias al momento de generar gráficos, métricas o indicadores.

Además, se recomienda que al importar el archivo en Power BI, la columna Postal Code sea tratada como texto, ya que no representa un valor numérico para cálculos, sino un identificador geográfico.

```
1 # Definimos la cantidad de decimales que queremos conservar
2 # en las columnas numéricas principales del dataset.
3 # Esto permite estandarizar los valores antes de exportarlos.
4
5 ROUNDING_CONFIG = {
6     "Sales": 2,
7     "Profit": 2,
8     "Discount": 4,
9     "Profit Margin": 4
10 }
11
12 # Aplicamos el redondeo sobre el dataset limpio.
13 # Cada columna se ajusta según la cantidad de decimales definida arriba.
14 df_clean = df_clean.round(ROUNDING_CONFIG)
```

```
1 # Exportamos el dataset limpio en formato Excel.
2 # El archivo se genera en el entorno temporal de Colab
3 # y luego se descarga automáticamente a la PC.
4
5 from google.colab import files
6
7 nombre_archivo = "df_sales_limpio_para_powerbi.xlsx"
8
9 df_clean.to_excel(nombre_archivo, index=False)
10
11 files.download(nombre_archivo)
```

7. Estadísticas descriptivas

Ahora analizamos variables numéricas principales. Esto permite ver magnitudes, dispersión y posibles valores extremos.

```
1 # Seleccionamos las variables numéricas más relevantes para el análisis.
2 # Con describe() obtenemos medidas resumen como promedio, mínimo, máximo,
3 # cuartiles y desviación estándar. La .T se usa para leer la tabla más fácilmente.
4
```

```

5
6 columnas_numericas = ['Sales', 'Quantity', 'Discount', 'Profit', 'Shipping Days', 'Profit Margin']
7 df_clean[columnas_numericas].describe().T

```

	count	mean	std	min	25%	50%	75%	max
Sales	9864.00	230.27	622.92	0.44	17.34	54.86	210.10	22638.48
Quantity	9864.00	3.79	2.23	1.00	2.00	3.00	5.00	14.00
Discount	9864.00	0.16	0.21	0.00	0.00	0.20	0.20	0.80
Profit	9864.00	28.65	234.51	-6599.98	1.73	8.67	29.37	8399.98
Shipping Days	9864.00	3.96	1.75	0.00	3.00	4.00	5.00	7.00
Profit Margin	9864.00	0.12	0.47	-2.75	0.07	0.27	0.36	0.50

```

1
2 # Sintesis:
3
4 # Calculamos percentiles para analizar la distribución de las variables principales.
5 # Esto nos ayuda a identificar valores típicos, extremos y posibles outliers.
6
7 # Miramos algunos percentiles para detectar valores extremos sin eliminarlos automáticamente.
8 percentiles = df_clean[['Sales', 'Profit', 'Quantity', 'Discount']].quantile([0.01, 0.05, 0.50, 0.95, 0.99])
9 percentiles

```

	Sales	Profit	Quantity	Discount
0.01	2.30	-319.58	1.00	0.00
0.05	4.98	-53.24	1.00	0.00
0.50	54.86	8.67	3.00	0.20
0.95	959.61	170.31	8.00	0.70
0.99	2489.13	581.17	11.00	0.80

Pasos siguientes: [Generar código con percentiles](#) [New interactive sheet](#)

Lectura inicial de las estadísticas

Se observa que **Sales** y **Profit** tienen mucha dispersión. Hay ventas de bajo monto y algunas operaciones muy grandes. También hay ganancias negativas, lo cual no necesariamente es un error: puede indicar descuentos agresivos, costos altos o productos poco rentables.

8. Análisis por categoría

Primero miramos ventas y ganancias por categoría. Esto ayuda a entender qué líneas del negocio aportan más facturación y rentabilidad.

```

1
2 # Agrupamos los datos por categoría para comparar ventas, ganancias,
3 # unidades vendidas y cantidad de operaciones por cada grupo de productos.
4
5 resumen_categoria = (
6     df_clean.groupby('Category')
7     .agg(
8         ventas_totales=('Sales', 'sum'),
9         ganancia_total=('Profit', 'sum'),
10        cantidad_vendida=('Quantity', 'sum'),
11        operaciones=('Order ID', 'nunique')
12     )
13     .sort_values('ventas_totales', ascending=False)
14 )
15
16 resumen_categoria

```

	ventas_totales	ganancia_total	cantidad_vendida	operaciones
Category				
Technology	803592.53	137998.24	6704	1505
Furniture	716409.05	17390.65	7790	1723
Office Supplies	702314.73	119472.92	22222	3658
Sin dato	49044.60	7716.34	698	190

Pasos siguientes:

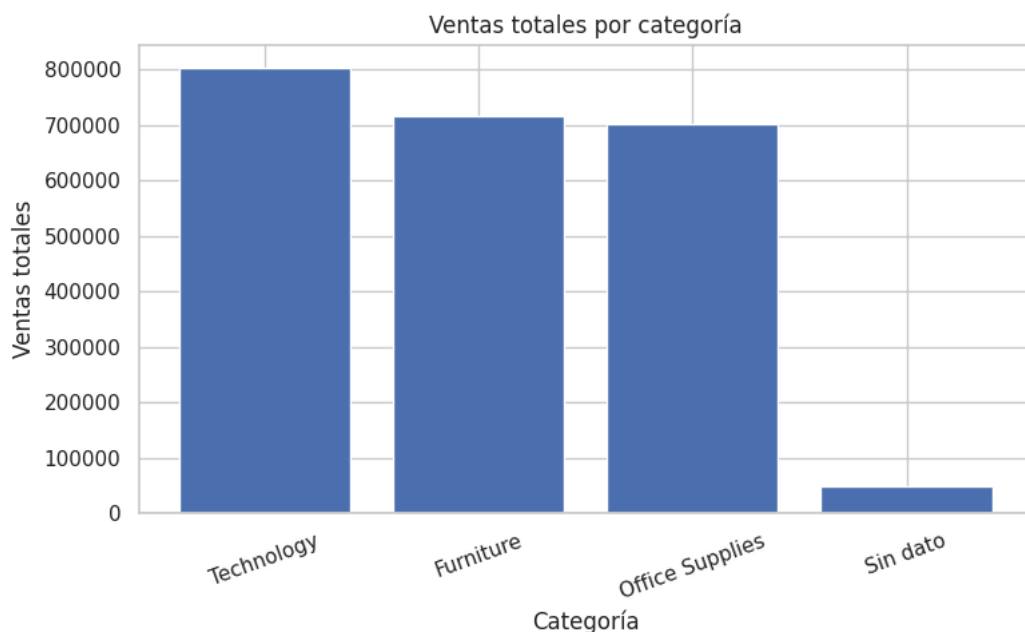
[Generar código con resumen_categoria](#)

[New interactive sheet](#)

```

1 # Convertimos el resumen en una tabla común para poder graficarlo.
2 # Luego hacemos un gráfico de barras para comparar visualmente
3 # las ventas totales de cada categoría.
4
5
6 plot_data = resumen_categoria.reset_index()
7 plt.figure(figsize=(8,5))
8 plt.bar(plot_data['Category'], plot_data['ventas_totales'])
9 plt.title('Ventas totales por categoría')
10 plt.xlabel('Categoría')
11 plt.ylabel('Ventas totales')
12 plt.xticks(rotation=20)
13 plt.tight_layout()
14 plt.show()

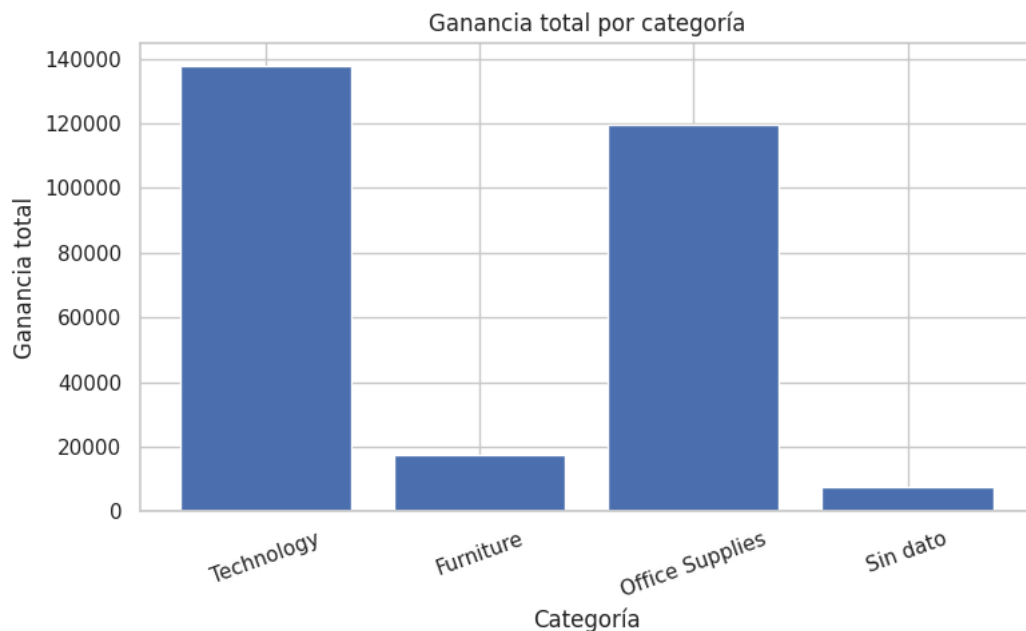
```



```

1 # Graficamos la ganancia total por categoría.
2 # Este gráfico permite comparar la rentabilidad de cada categoría,
3 # no solo el volumen de ventas.
4
5
6 plot_data = resumen_categoria.reset_index()
7 plt.figure(figsize=(8,5))
8 plt.bar(plot_data['Category'], plot_data['ganancia_total'])
9 plt.title('Ganancia total por categoría')
10 plt.xlabel('Categoría')
11 plt.ylabel('Ganancia total')
12 plt.xticks(rotation=20)
13 plt.tight_layout()
14 plt.show()

```



Lectura a primera vista: una categoría puede vender mucho, pero no necesariamente ser la más rentable. Por eso conviene mirar ventas y profit juntos.

9. Análisis por subcategoría

Bajamos un nivel más para detectar subcategorías que generan mucho ingreso y otras que pueden estar perjudicando el margen

```

1 # Agrupamos los datos por subcategoría para analizar con mayor detalle
2 # qué tipos de productos generan más ventas, ganancias y unidades vendidas.
3 # Luego ordenamos por ventas totales y mostramos las 10 subcategorías principales.
4
5
6 resumen_subcategoria = (
7     df_clean.groupby('Sub-Category')
8     .agg(
9         ventas_totales=('Sales', 'sum'),
10        ganancia_total=('Profit', 'sum'),
11        cantidad_vendida=('Quantity', 'sum')
12     )
13     .sort_values('ventas_totales', ascending=False)
14 )
15
16 resumen_subcategoria.head(10)

```

	ventas_totales	ganancia_total	cantidad_vendida
Sub-Category			
Chairs	320516.23	25959.75	2281
Phones	315661.39	42994.60	3113
Storage	216019.57	20555.22	3068
Tables	198714.75	-17424.50	1193
Binders	189767.68	25063.23	5735
Machines	180628.00	1927.46	428
Accessories	163302.27	41036.62	2873
Copiers	148528.03	55167.89	232
Bookcases	112166.77	-3593.11	851
Appliances	104938.75	18144.05	1682

Pasos siguientes:

[Generar código con resumen_subcategoria](#)

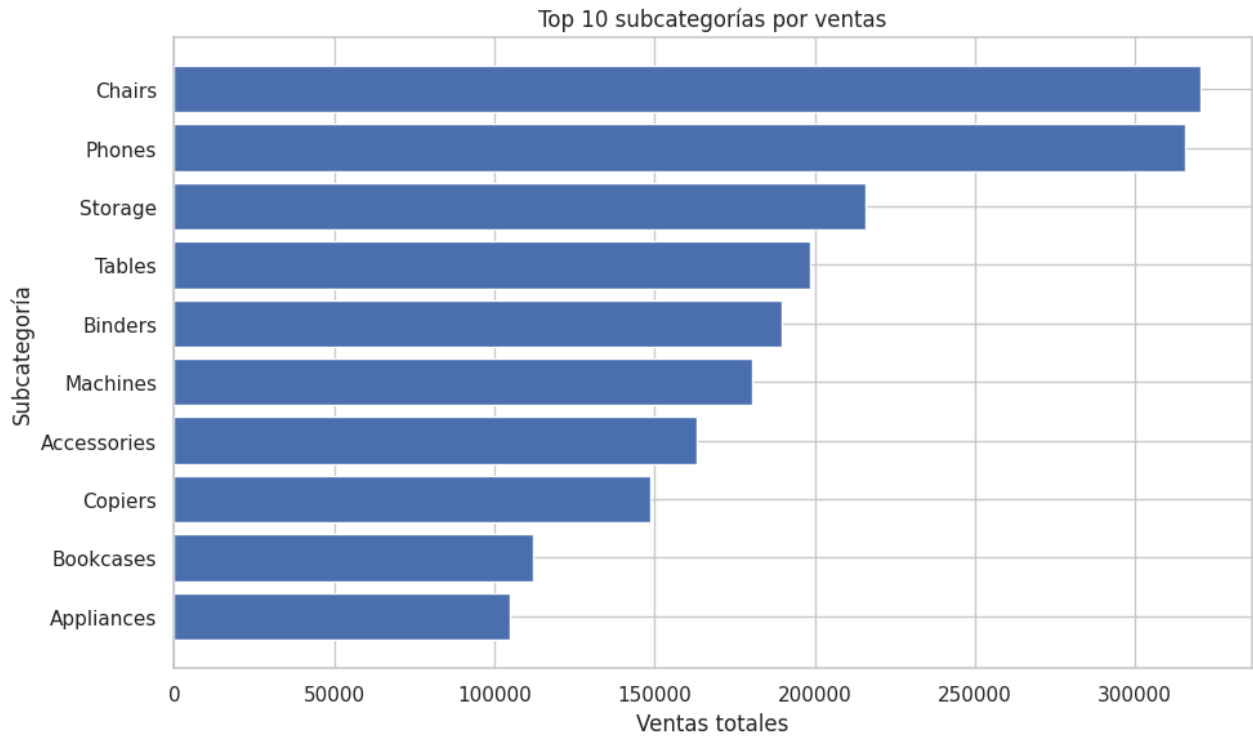
[New interactive sheet](#)

```

1 # Tomamos las 10 subcategorías con mayor venta y las graficamos.
2 # Usamos barras horizontales para mejorar la lectura de los nombres
3 # y ordenamos visualmente de mayor a menor.
4

```

```
5 # Visualizar las subcategorías más relevantes por facturación (ergo ingresos)
6
7 top_subcat_ventas = resumen_subcategoria.head(10).reset_index()
8
9 plt.figure(figsize=(10,6))
10 plt.barh(top_subcat_ventas['Sub-Category'], top_subcat_ventas['ventas_totales'])
11 plt.title('Top 10 subcategorías por ventas')
12 plt.xlabel('Ventas totales')
13 plt.ylabel('Subcategoría')
14 plt.gca().invert_yaxis()
15 plt.tight_layout()
16 plt.show()
```



```
1 # Ordenamos las subcategorías por ganancia total de menor a mayor.
2 # Así identificamos las 10 subcategorías con peor resultado económico.
3
4 peores_subcat_profit = resumen_subcategoria.sort_values('ganancia_total').head(10).reset_index()
5 peores_subcat_profit
```

	Sub-Category	ventas_totales	ganancia_total	cantidad_vendida
0	Tables	198714.75	-17424.50	1193
1	Bookcases	112166.77	-3593.11	851
2	Supplies	45647.66	-988.35	633
3	Fasteners	2962.41	928.86	888
4	Machines	180628.00	1927.46	428
5	Labels	12351.15	5484.72	1367
6	Art	26459.90	6343.15	2893
7	Envelopes	16006.94	6782.95	887
8	Sin dato	52709.58	8417.82	840
9	Furnishings	89278.14	12906.75	3454

Pasos siguientes:

[Generar código con peores_subcat_profit](#)

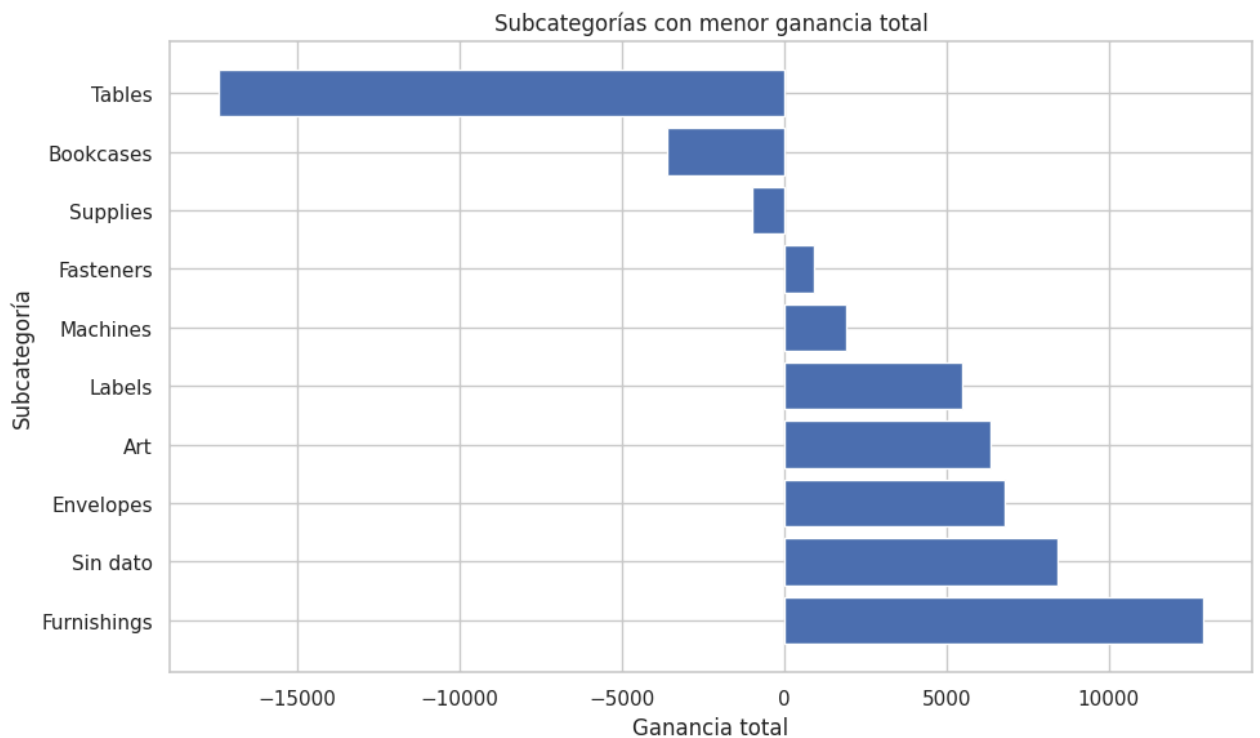
[New interactive sheet](#)

```
1 # Graficamos las subcategorías con menor ganancia total.
2 # Esto permite identificar visualmente productos o líneas que pueden estar generando baja rentabilidad o pérdidas.
3
4
5 plt.figure(figsize=(10,6))
6 plt.barh(peores_subcat_profit['Sub-Category'], peores_subcat_profit['ganancia_total'])
7 plt.title('Subcategorías con menor ganancia total')
8 plt.xlabel('Ganancia total')
9 plt.ylabel('Subcategoría')
```

```

10 plt.gca().invert_yaxis()
11 plt.tight_layout()
12 plt.show()

```



Este gráfico es útil porque muestra dónde puede haber problemas de rentabilidad. No alcanza con vender mucho: algunas subcategorías pueden estar generando pérdida.

10. Análisis por región y segmento

Ahora miramos el negocio por ubicación y tipo de cliente.

```

1 # Agrupamos los datos por región para analizar el desempeño geográfico.
2 # Calculamos ventas, ganancias y cantidad de órdenes únicas por cada región.
3
4
5 resumen_region = (
6     df_clean.groupby('Region')
7     .agg(
8         ventas_totales=('Sales', 'sum'),
9         ganancia_total=('Profit', 'sum'),
10        operaciones=('Order ID', 'nunique')
11    )
12    .sort_values('ventas_totales', ascending=False)
13 )
14
15 resumen_region

```

	ventas_totales	ganancia_total	operaciones
Region			
West	708155.64	105680.90	1580
East	647812.03	87425.51	1367
Central	490819.51	39095.07	1153
South	382350.88	45693.26	814
Sin dato	42222.85	4683.41	199

Pasos siguientes: [Generar código con resumen_region](#) [New interactive sheet](#)

```

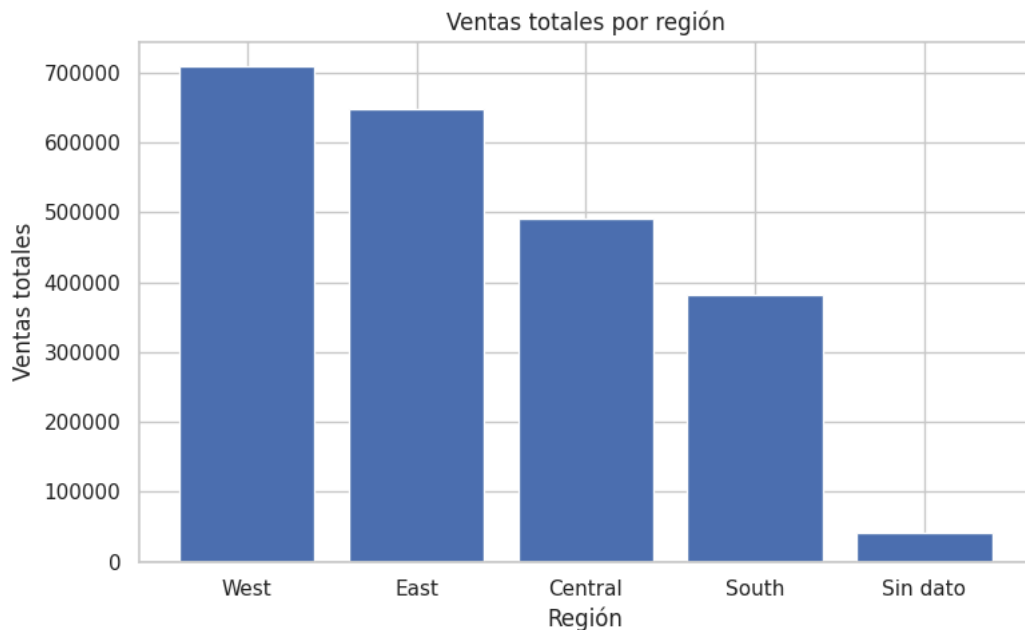
1 # Graficamos las ventas totales por región.
2 # Esto permite comparar visualmente qué zonas concentran mayor facturación.
3
4

```

```

5 plot_data = resumen_region.reset_index()
6 plt.figure(figsize=(8,5))
7 plt.bar(plot_data['Region'], plot_data['ventas_totales'])
8 plt.title('Ventas totales por región')
9 plt.xlabel('Región')
10 plt.ylabel('Ventas totales')
11 plt.tight_layout()
12 plt.show()

```



```

1 # Agrupamos los datos por segmento de cliente para comparar su desempeño.
2 # Calculamos ventas totales, ganancia total, cantidad de órdenes únicas
3 # y venta promedio por registro.
4
5 resumen_segmento = (
6     df_clean.groupby('Segment')
7     .agg(
8         ventas_totales=('Sales', 'sum'),
9         ganancia_total=('Profit', 'sum'),
10        operaciones=('Order ID', 'nunique'),
11        ticket_promedio=('Sales', 'mean')
12    )
13    .sort_values('ventas_totales', ascending=False)
14 )
15
16 resumen_segmento

```

	ventas_totales	ganancia_total	operaciones	ticket_promedio
Segment				
Consumer	1117630.07	127256.95	2533	223.17
Corporate	684946.31	90161.85	1489	234.57
Home Office	418696.44	57622.00	887	243.85
Sin dato	50088.09	7537.35	215	228.71

Pasos siguientes:

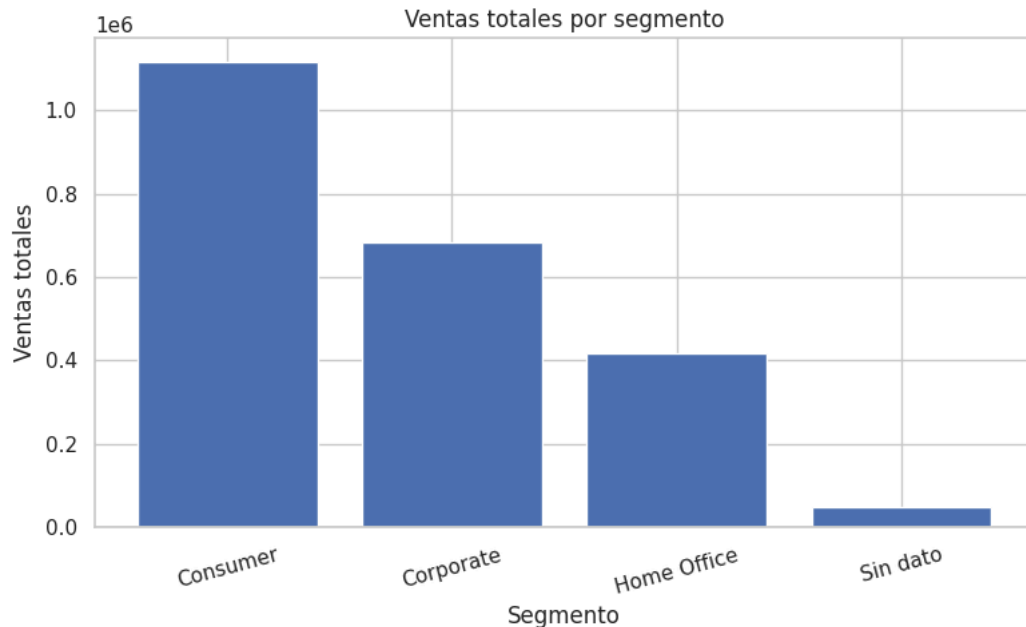
[Generar código con resumen_segmento](#)

[New interactive sheet](#)

```

1 # Graficamos las ventas totales por segmento de cliente.
2 # Esto nos permite comparar visualmente qué segmento aporta más facturación.
3
4
5 plot_data = resumen_segmento.reset_index()
6 plt.figure(figsize=(8,5))
7 plt.bar(plot_data['Segment'], plot_data['ventas_totales'])
8 plt.title('Ventas totales por segmento')
9 plt.xlabel('Segmento')
10 plt.ylabel('Ventas totales')
11 plt.xticks(rotation=15)
12 plt.tight_layout()
13 plt.show()

```

11. Evolución mensual de ventas

Como tenemos fechas, podemos ver cómo cambian las ventas a lo largo del tiempo.

```

1 # Agrupamos los datos por año-mes del pedido para analizar la evolución
  temporal.
2 # Calculamos ventas, ganancias y cantidad de órdenes únicas por cada mes.
3 # Luego ordenamos cronológicamente para poder graficar correctamente la serie.
4
5
6 ventas_mensuales = (
7     df_clean.groupby('Order Year-Month')
8     .agg(
9         ventas_totales=('Sales', 'sum'),
10        ganancia_total=('Profit', 'sum'),
11        operaciones=('Order ID', 'nunique')
12     )
13     .reset_index()
14     .sort_values('Order Year-Month')
15 )
16
17 ventas_mensuales.head()

```

	Order Year-Month	ventas_totales	ganancia_total	operaciones
0	2014-01	14236.90	2450.17	32
1	2014-02	4519.92	862.30	28
2	2014-03	55321.46	452.69	70
3	2014-04	28295.35	3488.83	66
4	2014-05	23422.97	2716.19	68

Pasos siguientes:

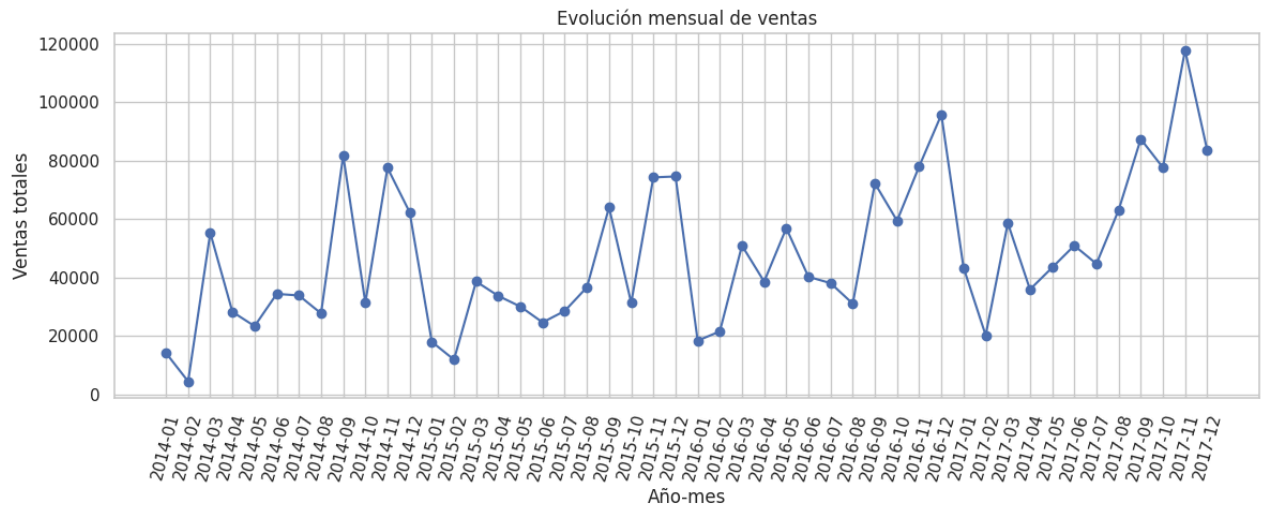
[Generar código con ventas_mensuales](#)

[New interactive sheet](#)

```

1 # Graficamos la evolución mensual de ventas.
2 # Usamos un gráfico de línea porque queremos observar cambios en el tiempo,
3 # como tendencias, picos o caídas en determinados meses.
4
5 plt.figure(figsize=(12,5))
6 plt.plot(ventas_mensuales['Order Year-Month'], ventas_mensuales
7         ['ventas_totales'], marker='o')
8 plt.title('Evolución mensual de ventas')
9 plt.xlabel('Año-mes')
10 plt.ylabel('Ventas totales')
11 plt.xticks(rotation=75)
12 plt.tight_layout()
13 plt.show()

```



12. Descuentos y rentabilidad

Los descuentos pueden ayudar a vender, pero también pueden afectar la ganancia. Revisamos la relación entre descuento y profit.

```

1 # Agrupamos los datos por nivel de descuento para analizar su relación
2 # con las ventas, la ganancia y la cantidad de registros.
3 # Esto ayuda a observar si los descuentos altos se asocian con menor rentabilidad.
4
5
6
7 resumen_descuento = (
8     df_clean.groupby('Discount')
9     .agg(
10         ventas_totales=('Sales', 'sum'),
11         ganancia_total=('Profit', 'sum'),
12         operaciones=('Order ID', 'count')
13     )
14     .reset_index()
15     .sort_values('Discount')
16 )
17
18 resumen_descuento

```

	Discount	ventas_totales	ganancia_total	operaciones
0	0.00	1072166.53	316342.43	4737
1	0.10	53511.06	8886.17	93
2	0.15	26754.53	1437.89	50
3	0.20	760509.45	89586.74	3612
4	0.30	102867.39	-10303.41	224
5	0.32	14493.45	-2391.16	27
6	0.40	113022.95	-22206.69	200
7	0.45	5484.98	-2493.12	11
8	0.50	58918.53	-20506.49	66
9	0.60	6636.92	-5942.49	137
10	0.70	40068.52	-39350.92	411
11	0.80	16926.60	-30480.80	296

Pasos siguientes:

[Generar código con resumen_descuento](#)

[New interactive sheet](#)

```

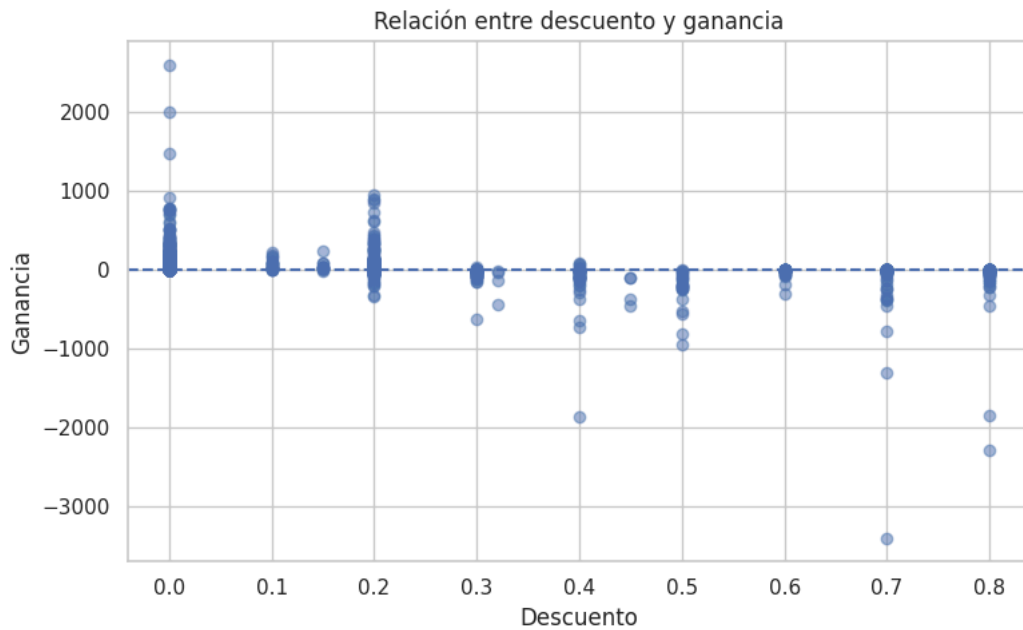
1 # Tomamos una muestra para que el gráfico no quede sobrecargado.
2 # Luego graficamos la relación entre descuento y ganancia.
3 # La línea horizontal en cero ayuda a distinguir operaciones con ganancia y con pérdida.
4

```

```

5
6 muestra = df_clean.sample(min(2500, len(df_clean)), random_state=42)
7 plt.figure(figsize=(8,5))
8 plt.scatter(muestra['Discount'], muestra['Profit'], alpha=0.5)
9 plt.axhline(0, linestyle='--')
10 plt.title('Relación entre descuento y ganancia')
11 plt.xlabel('Descuento')
12 plt.ylabel('Ganancia')
13 plt.tight_layout()
14 plt.show()

```



13. Tiempo de envío

El campo **Shipping Days** permite ver la demora entre la fecha de pedido y la fecha de envío.

```

1 # Agrupamos los datos por modo de envío para comparar tiempos de entrega.
2 # Calculamos demora promedio, demora mediana y cantidad de registros por cada modalidad.
3 # Luego ordenamos desde el modo con menor demora promedio al de mayor demora.
4
5
6 resumen_envio = (
7     df_clean.groupby('Ship Mode')
8     .agg(
9         demora_promedio=('Shipping Days', 'mean'),
10        demora_mediana=('Shipping Days', 'median'),
11        operaciones=('Order ID', 'count')
12    )
13    .sort_values('demora_promedio')
14 )
15
16 resumen_envio

```

	demora_promedio	demora_mediana	operaciones
Ship Mode			
Same Day	0.05	0.00	517
First Class	2.18	2.00	1472
Second Class	3.24	3.00	1888
Sin dato	3.73	4.00	224
Standard Class	5.01	5.00	5763

Pasos siguientes: [Generar código con resumen_envio](#) [New interactive sheet](#)

```

1 # Creamos un boxplot para comparar la distribución de los días de envío
2 # según cada modo de envío. Este gráfico permite observar medianas,
3 # dispersión y posibles valores atípicos.
4
5

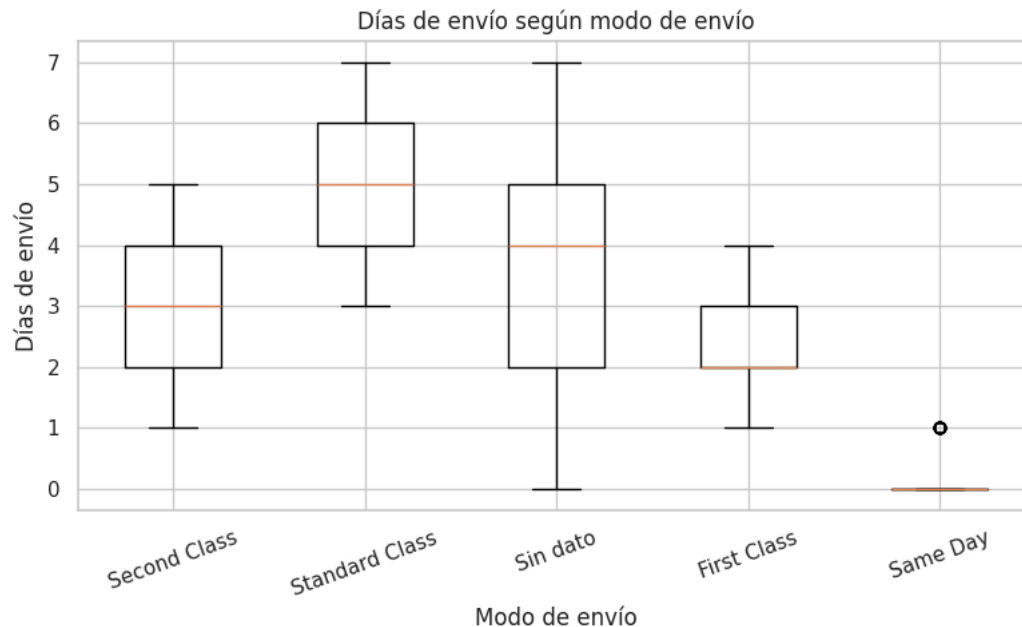
```

```

6 orden_ship = df_clean['Ship Mode'].dropna().unique()
7 datos_box = [df_clean.loc[df_clean['Ship Mode'] == modo, 'Shipping Days'] for modo in orden_ship]
8
9 plt.figure(figsize=(8,5))
10 plt.boxplot(datos_box, labels=orden_ship)
11 plt.title('Días de envío según modo de envío')
12 plt.xlabel('Modo de envío')
13 plt.ylabel('Días de envío')
14 plt.xticks(rotation=20)
15 plt.tight_layout()
16 plt.show()

```

/tmp/ipykernel_7059/755199894.py:10: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' in 3.8; please update your code. Use of the 'labels' parameter has been deprecated.
plt.boxplot(datos_box, labels=orden_ship)



✓ 14. Análisis RFM · Segmentación de clientes

RFM significa:

- Recency: días desde la última compra del cliente.
- Frequency: cantidad de órdenes realizadas.
- Monetary: gasto total acumulado por cliente.

```

1
2 1. # Base para análisis RFM
3
4 df_rfm_base = df_clean.copy()
5
6 # Nos aseguramos de que las columnas necesarias tengan el tipo correcto
7 df_rfm_base['Order Date'] = pd.to_datetime(df_rfm_base['Order Date'], errors='coerce')
8 df_rfm_base['Sales'] = pd.to_numeric(df_rfm_base['Sales'], errors='coerce')
9
10 # Eliminamos registros que no sirven para calcular RFM
11 df_rfm_base = df_rfm_base.dropna(
12     subset=['Customer ID', 'Order ID', 'Order Date', 'Sales']
13 ).copy()
14
15 # Fecha de referencia: última fecha de compra disponible
16 fecha_referencia = df_rfm_base['Order Date'].max()
17
18
19 # 2. Cálculo RFM por cliente
20
21
22 rfm = (
23     df_rfm_base
24     .groupby('Customer ID')
25     .agg(
26         Recency=('Order Date', lambda x: (fecha_referencia - x.max()).days),
27         Frequency=('Order ID', 'nunique'),
28         Monetary=('Sales', 'sum')
29     )
30     .reset_index()
31 )

```

```

32
33 rfm = rfm.rename(columns={'Customer ID': 'ID_cliente'})
34
35 rfm.head()

```

	ID_cliente	Recency	Frequency	Monetary	
0	AA-10315	184	5	5563.56	
1	AA-10375	19	8	1022.12	
2	AA-10480	259	4	1790.51	
3	AA-10645	55	6	5086.93	
4	AB-10015	415	3	643.21	

Pasos siguientes:

[Generar código con rfm](#)

[New interactive sheet](#)

```

1 # Puntuaciones RFM
2
3 # Cada dimensión recibe un puntaje de 1 a 5.
4 # En Recency, menor cantidad de días es mejor.
5 # En Frequency, más órdenes es mejor.
6 # En Monetary, más gasto acumulado es mejor.
7
8 rfm['R_Score'] = pd.qcut(
9     rfm['Recency'].rank(method='first'),
10    q=5,
11    labels=[5, 4, 3, 2, 1]
12 )
13
14 rfm['F_Score'] = pd.qcut(
15     rfm['Frequency'].rank(method='first'),
16     q=5,
17     labels=[1, 2, 3, 4, 5]
18 )
19
20 rfm['M_Score'] = pd.qcut(
21     rfm['Monetary'].rank(method='first'),
22     q=5,
23     labels=[1, 2, 3, 4, 5]
24 )
25
26 # Convertimos los scores a enteros
27 rfm['R_Score'] = rfm['R_Score'].astype(int)
28 rfm['F_Score'] = rfm['F_Score'].astype(int)
29 rfm['M_Score'] = rfm['M_Score'].astype(int)
30
31 # Score total
32 rfm['RFM_Score'] = rfm[['R_Score', 'F_Score', 'M_Score']].sum(axis=1)
33
34 # Código RFM combinado, por ejemplo 555, 445, etc.
35 rfm['RFM_Code'] = (
36     rfm['R_Score'].astype(str) +
37     rfm['F_Score'].astype(str) +
38     rfm['M_Score'].astype(str)
39 )
40
41 rfm.head()

```

	ID_cliente	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score	RFM_Code	
0	AA-10315	184	5	5563.56	2	2	5	9	225	
1	AA-10375	19	8	1022.12	5	4	2	11	542	
2	AA-10480	259	4	1790.51	1	1	3	5	113	
3	AA-10645	55	6	5086.93	3	3	5	11	335	
4	AB-10015	415	3	643.21	1	1	1	3	111	

Pasos siguientes:

[Generar código con rfm](#)

[New interactive sheet](#)

```

1 #Segmentación RFM
2
3 # Esta segmentación está adaptada al enfoque del archivo RFM_Analisis.pdf:
4 # Champions, Loyal, At Risk, Needs Attention, New Customers, Lost y Potential.
5
6 def segmentar_rfm(row):
7     r = row['R_Score']

```

```

8   f = row['F_Score']
9   m = row['M_Score']
10
11  # Clientes recientes, frecuentes y de alto valor
12  if r >= 4 and f >= 4 and m >= 4:
13      return 'Champions'
14
15  # Compran con buena frecuencia y valor, aunque no siempre son los más recientes
16  elif r >= 3 and f >= 4 and m >= 3:
17      return 'Loyal'
18
19  # Clientes de alto valor o frecuencia, pero que llevan mucho tiempo sin comprar
20  elif r <= 2 and f >= 4 and m >= 3:
21      return 'At Risk'
22
23  # Clientes con cierto valor, pero con señales de enfriamiento
24  elif r <= 3 and f >= 3 and m >= 3:
25      return 'Needs Attention'
26
27  # Clientes recientes, pero todavía con baja frecuencia o bajo gasto acumulado
28  elif r >= 4 and f <= 3:
29      return 'New Customers'
30
31  # Clientes con baja recencia, baja frecuencia y bajo valor
32  elif r <= 2 and f <= 3:
33      return 'Lost'
34
35  # Clientes intermedios con posibilidad de crecer
36  else:
37      return 'Potential'
38
39
40 rfm['Segmento_RFM'] = rfm.apply(segmentar_rfm, axis=1)
41
42 rfm.head()

```

	ID_cliente	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score	RFM_Code	Segmento_RFM	
0	AA-10315	184	5	5563.56	2	2	5	9	225	Lost	
1	AA-10375	19	8	1022.12	5	4	2	11	542	Potential	
2	AA-10480	259	4	1790.51	1	1	3	5	113	Lost	
3	AA-10645	55	6	5086.93	3	3	5	11	335	Needs Attention	
4	AB-10015	415	3	643.21	1	1	1	3	111	Lost	

Pasos siguientes:

[Generar código con rfm](#)

[New interactive sheet](#)

```

1 # Resumen por segmento RFM
2
3
4 revenue_total = rfm['Monetary'].sum()
5 clientes_total = rfm['ID_cliente'].nunique()
6
7 acciones_segmento = {
8     'Champions': 'Fidelizar',
9     'Loyal': 'Upsell',
10    'At Risk': 'Reactivar urgente',
11    'Needs Attention': 'Ofertas personalizadas',
12    'New Customers': 'Onboarding',
13    'Lost': 'Win-back o baja',
14    'Potential': '2a compra'
15 }
16
17 orden_segmentos = [
18     'Champions',
19     'Loyal',
20     'At Risk',
21     'Needs Attention',
22     'New Customers',
23     'Lost',
24     'Potential'
25 ]
26
27 resumen_rfm = (
28     rfm.groupby('Segmento_RFM')
29     .agg(
30         Clientes=('ID_cliente', 'count'),
31         Revenue=('Monetary', 'sum'),
32         Recencia=('Recency', 'mean'),

```

```

33     Frec=('Frequency', 'mean'),
34     Ticket=('Monetary', 'mean')
35 )
36 .reindex(orden_segmentos)
37 .fillna(0)
38 )
39
40 resumen_rfm['% Clientes'] = resumen_rfm['Clientes'] / clientes_total * 100
41 resumen_rfm['% Rev.'] = resumen_rfm['Revenue'] / revenue_total * 100
42 resumen_rfm['Acción'] = resumen_rfm.index.map(acciones_segmento)
43
44 # Reordenamos columnas para que quede como en el PDF
45 resumen_rfm = resumen_rfm[
46     ['Clientes', '% Clientes', '% Rev.', 'Recencia', 'Frec', 'Ticket', 'Revenue', 'Acción']
47 ]
48
49 # Redondeo para presentación
50 resumen_rfm_presentacion = resumen_rfm.copy()
51 resumen_rfm_presentacion['% Clientes'] = resumen_rfm_presentacion['% Clientes'].round(1)
52 resumen_rfm_presentacion['% Rev.'] = resumen_rfm_presentacion['% Rev.'].round(1)
53 resumen_rfm_presentacion['Recencia'] = resumen_rfm_presentacion['Recencia'].round(0).astype(int)
54 resumen_rfm_presentacion['Frec'] = resumen_rfm_presentacion['Frec'].round(1)
55 resumen_rfm_presentacion['Ticket'] = resumen_rfm_presentacion['Ticket'].round(0)
56 resumen_rfm_presentacion['Revenue'] = resumen_rfm_presentacion['Revenue'].round(0)
57
58 resumen_rfm_presentacion

```

	Clientes	% Clientes	% Rev.	Recencia	Frec	Ticket	Revenue	Acción
Segmento_RFM								
Champions	103	13.00	23.80	24	9.30	5241.00	539832.00	Fidelizar
Loyal	95	12.00	14.10	56	8.60	3376.00	320757.00	Upsell
At Risk	64	8.10	14.20	208	8.70	5049.00	323158.00	Reactivar urgente
Needs Attention	55	6.90	9.00	190	6.10	3724.00	204793.00	Ofertas personalizadas
New Customers	151	19.00	15.90	26	5.00	2387.00	360500.00	Onboarding
Lost	201	25.30	13.20	355	4.00	1489.00	299295.00	Win-back o baja
Potential	124	15.60	9.80	83	6.00	1799.00	223026.00	2a compra

Pasos siguientes: [Generar código con resumen_rfm_presentacion](#) [New interactive sheet](#)

```

1
2 # 19. Indicadores principales del análisis RFM
3
4 clientes_champions_loyal = resumen_rfm.loc[['Champions', 'Loyal'], 'Clientes'].sum()
5 revenue_champions_loyal = resumen_rfm.loc[['Champions', 'Loyal'], 'Revenue'].sum()
6 porcentaje_revenue_champions_loyal = revenue_champions_loyal / revenue_total * 100
7
8 clientes_at_risk = resumen_rfm.loc['At Risk', 'Clientes']
9 revenue_at_risk = resumen_rfm.loc['At Risk', 'Revenue']
10 porcentaje_revenue_at_risk = revenue_at_risk / revenue_total * 100
11
12 clientes_lost = resumen_rfm.loc['Lost', 'Clientes']
13 recencia_lost = resumen_rfm.loc['Lost', 'Recencia']
14
15 anio_min = df_rfm_base['Order Date'].dt.year.min()
16 anio_max = df_rfm_base['Order Date'].dt.year.max()
17
18 print("Clientes analizados:", clientes_total)
19 print("Años del historial:", anio_min, "-", anio_max)
20 print("Champions + Loyal:", int(clientes_champions_loyal), "clientes")
21 print("% Revenue Champions + Loyal:", round(porcentaje_revenue_champions_loyal, 1), "%")
22 print("At Risk:", int(clientes_at_risk), "clientes")
23 print("% Revenue At Risk:", round(porcentaje_revenue_at_risk, 1), "%")
24 print("Lost:", int(clientes_lost), "clientes")
25 print("Recencia promedio Lost:", round(recencia_lost, 0), "días sin comprar")

```

Clientes analizados: 793
 Años del historial: 2014 - 2017
 Champions + Loyal: 198 clientes
 % Revenue Champions + Loyal: 37.9 %
 At Risk: 64 clientes
 % Revenue At Risk: 14.2 %
 Lost: 201 clientes
 Recencia promedio Lost: 355.0 días sin comprar

```

1 # Indicadores principales del análisis RFM
2
3 clientes_champions_loyal = resumen_rfm.loc[['Champions', 'Loyal'], 'Clientes'].sum()
4 revenue_champions_loyal = resumen_rfm.loc[['Champions', 'Loyal'], 'Revenue'].sum()
5 porcentaje_revenue_champions_loyal = revenue_champions_loyal / revenue_total * 100
6
7 clientes_at_risk = resumen_rfm.loc['At Risk', 'Clientes']
8 revenue_at_risk = resumen_rfm.loc['At Risk', 'Revenue']
9 porcentaje_revenue_at_risk = revenue_at_risk / revenue_total * 100
10
11 clientes_lost = resumen_rfm.loc['Lost', 'Clientes']
12 recencia_lost = resumen_rfm.loc['Lost', 'Recencia']
13
14 anio_min = df_rfm_base['Order Date'].dt.year.min()
15 anio_max = df_rfm_base['Order Date'].dt.year.max()
16
17 print("Clientes analizados:", clientes_total)
18 print("Años del historial:", anio_min, "-", anio_max)
19 print("Champions + Loyal:", int(clientes_champions_loyal), "clientes")
20 print("% Revenue Champions + Loyal:", round(porcentaje_revenue_champions_loyal, 1), "%")
21 print("At Risk:", int(clientes_at_risk), "clientes")
22 print("% Revenue At Risk:", round(porcentaje_revenue_at_risk, 1), "%")
23 print("Lost:", int(clientes_lost), "clientes")
24 print("Recencia promedio Lost:", round(recencia_lost, 0), "días sin comprar")

```

Clientes analizados: 793
 Años del historial: 2014 - 2017
 Champions + Loyal: 198 clientes
 % Revenue Champions + Loyal: 37.9 %
 At Risk: 64 clientes
 % Revenue At Risk: 14.2 %
 Lost: 201 clientes
 Recencia promedio Lost: 355.0 días sin comprar

```

1
2 # Distribución por segmento: % Clientes y % Revenue
3
4 # Este gráfico reproduce la lógica del archivo RFM_Analisis.pdf:
5 # izquierda = distribución de clientes por segmento
6 # derecha = distribución del revenue por segmento
7
8 colores_segmentos = {
9     'Champions': '#1b9e77',
10    'Loyal': '#377eb8',
11    'Needs Attention': '#f39c12',
12    'At Risk': '#e41a1c',
13    'New Customers': '#8bc34a',
14    'Lost': '#7f7f7f',
15    'Potential': '#6a5acd'
16 }
17
18 # IMPORTANTE:
19 # resumen_rfm tiene los segmentos en el índice.
20 # Por eso primero convertimos el índice en una columna.
21 plot_rfm = resumen_rfm.copy()
22 plot_rfm.index.name = 'Segmento_RFM'
23 plot_rfm = plot_rfm.reset_index()
24
25 # Por seguridad, recalculamos los porcentajes si no existieran
26 if '% Clientes' not in plot_rfm.columns:
27     plot_rfm['% Clientes'] = plot_rfm['Clientes'] / plot_rfm['Clientes'].sum() * 100
28
29 if '% Rev.' not in plot_rfm.columns:
30     plot_rfm['% Rev.'] = plot_rfm['Revenue'] / plot_rfm['Revenue'].sum() * 100
31
32 # Quitamos segmentos sin clientes para evitar problemas visuales
33 plot_rfm = plot_rfm[plot_rfm['Clientes'] > 0].copy()
34
35 # Creamos figura con dos gráficos tipo dona
36 fig, axes = plt.subplots(1, 2, figsize=(13, 6))
37
38 # Gráfico 1: % Clientes
39 axes[0].pie(
40     plot_rfm['% Clientes'],
41     labels=plot_rfm['Segmento_RFM'],
42     autopct='%1.0f%%',
43     startangle=90,
44     colors=[colores_segmentos.get(seg, '#cccccc') for seg in plot_rfm['Segmento_RFM']],
45     wedgeprops=dict(width=0.45)
46 )
47
48 axes[0].set_title('% Clientes por segmento')

```

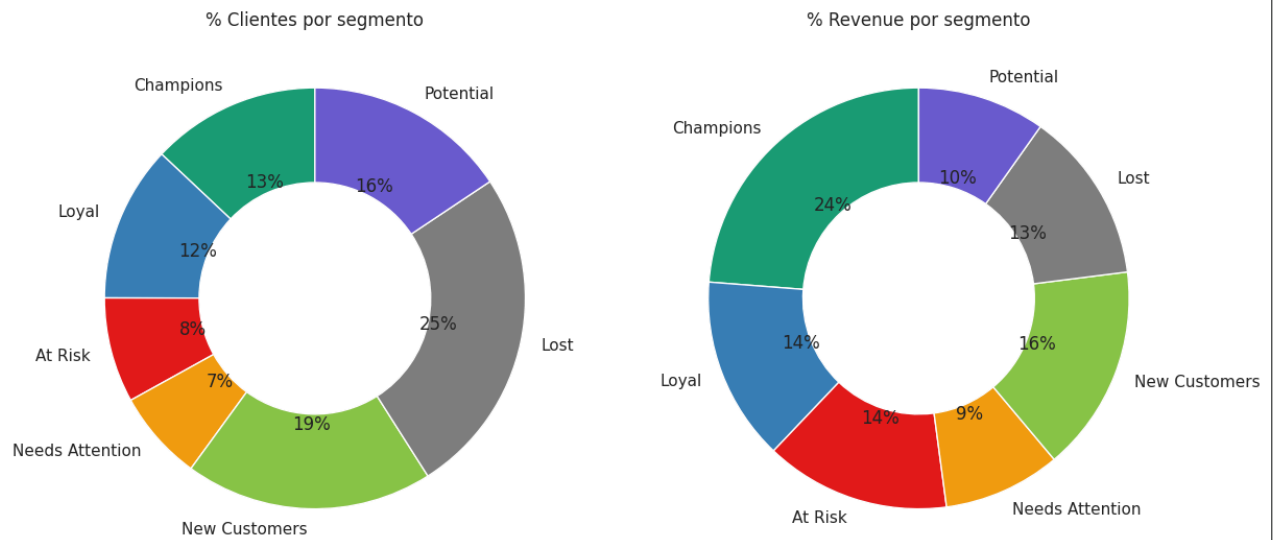


```

49
50 # Gráfico 2: % Revenue
51 axes[1].pie(
52     plot_rfm['% Rev.'],
53     labels=plot_rfm['Segmento_RFM'],
54     autopct='%1.0f%%',
55     startangle=90,
56     colors=[colores_segmentos.get(seg, '#cccccc') for seg in plot_rfm['Segmento_RFM']],
57     wedgeprops=dict(width=0.45)
58 )
59
60 axes[1].set_title('% Revenue por segmento')
61
62 plt.suptitle('Distribución por segmento RFM', fontsize=16, fontweight='bold')
63 plt.tight_layout()
64 plt.show()

```

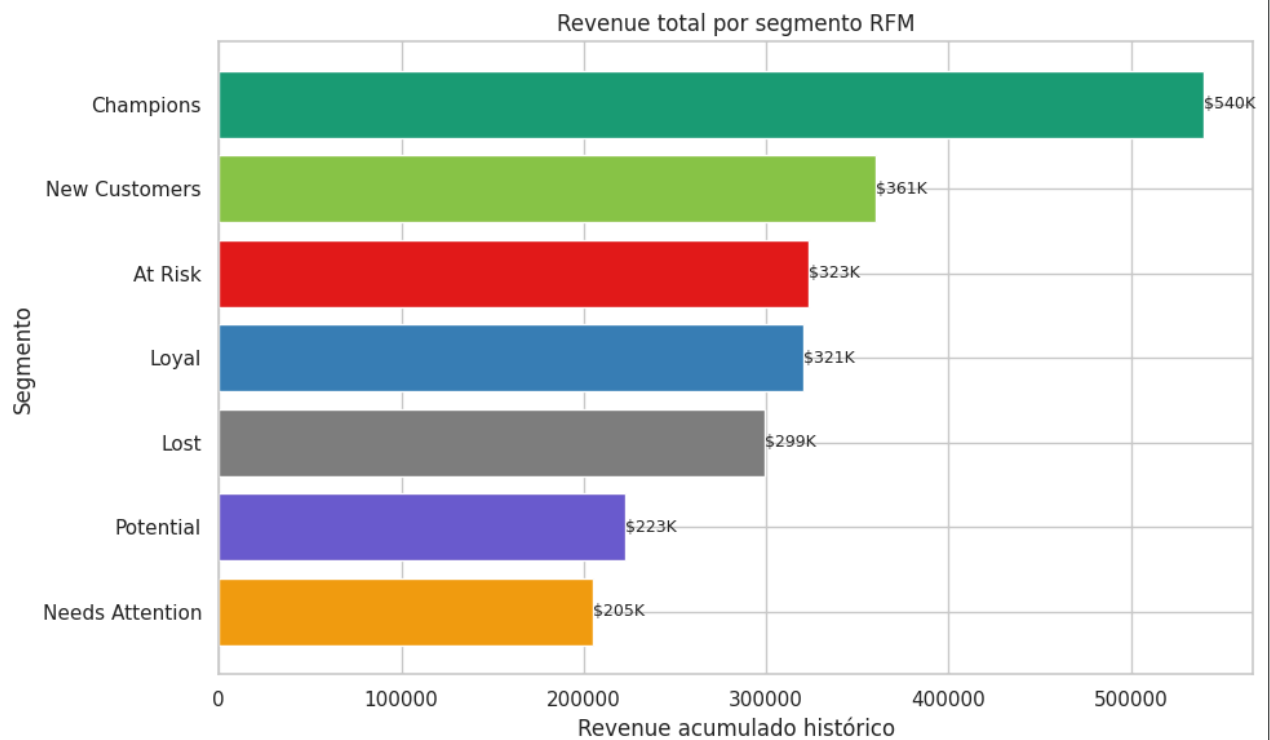
Distribución por segmento RFM



```

1
2 # Revenue total por segmento
3
4 revenue_segmento = (
5     resumen_rfm
6     .sort_values('Revenue', ascending=True)
7     .reset_index()
8 )
9
10 plt.figure(figsize=(10, 6))
11
12 plt.barh(
13     revenue_segmento['Segmento_RFM'],
14     revenue_segmento['Revenue'],
15     color=[colores_segmentos.get(seg, '#cccccc') for seg in revenue_segmento['Segmento_RFM']]
16 )
17
18 plt.title('Revenue total por segmento RFM')
19 plt.xlabel('Revenue acumulado histórico')
20 plt.ylabel('Segmento')
21
22 # Etiquetas al final de cada barra
23 for i, valor in enumerate(revenue_segmento['Revenue']):
24     plt.text(
25         valor,
26         i,
27         f'${valor/1000:.0f}K',
28         va='center',
29         fontsize=9
30     )
31
32 plt.tight_layout()
33 plt.show()

```



```

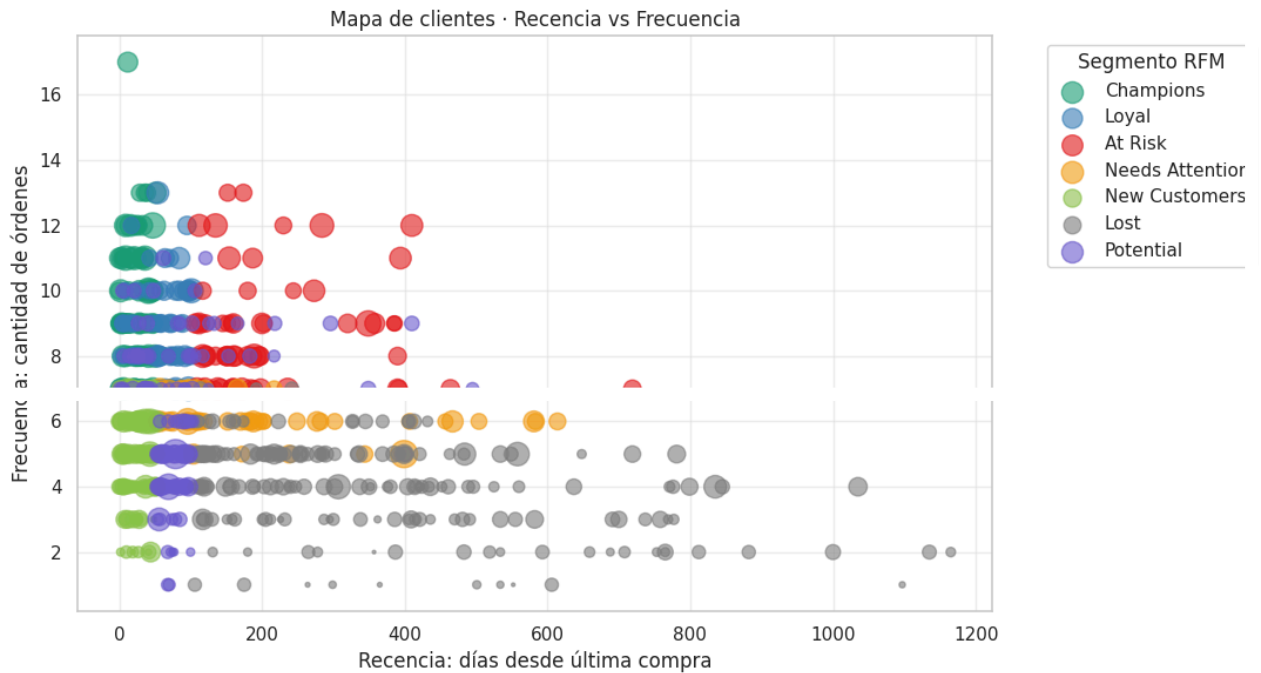
1
2 # Mapa de clientes · Recencia vs Frecuencia
3
4
5 plt.figure(figsize=(11, 6))
6
7 for segmento in orden_segmentos:
8     subset = rfm[rfm['Segmento_RFM'] == segmento]
9
10     plt.scatter(
11         subset['Recency'],
12         subset['Frequency'],
13         s=np.sqrt(subset['Monetary']) * 2,
14         alpha=0.6,
15         label=segmento,
16         color=colores_segmentos.get(segmento, '#cccccc')

```

```

17 )
18
19 plt.title('Mapa de clientes · Recencia vs Frecuencia')
20 plt.xlabel('Recencia: días desde última compra')
21 plt.ylabel('Frecuencia: cantidad de órdenes')
22 plt.legend(title='Segmento RFM', bbox_to_anchor=(1.05, 1), loc='upper left')
23 plt.grid(True, alpha=0.3)
24 plt.tight_layout()
25 plt.show()

```



Análisis RFM · Segmentación de clientes

El análisis RFM permite clasificar a los clientes a partir de tres dimensiones comerciales:

- **Recencia:** cantidad de días desde la última compra.
- **Frecuencia:** cantidad de órdenes realizadas por el cliente.
- **Monetario:** gasto total acumulado por cliente.

A cada dimensión se le asignó un puntaje del 1 al 5. En el caso de **Recency**, los clientes con compras más recientes reciben mayor puntaje. En **Frequency** y **Monetary**, los clientes con más órdenes y mayor gasto acumulado reciben mayor valoración.

A partir de la combinación de estos puntajes se construyeron siete segmentos:

- **Champions:** clientes recientes, frecuentes y de alto valor.
- **Loyal:** clientes con buena frecuencia y buen nivel de compra.
- **At Risk:** clientes valiosos que hace mucho tiempo no compran.
- **Needs Attention:** clientes que todavía tienen valor, pero muestran señales de enfriamiento.